ComplyDesk — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and keeping layers separate

A generate→verify→revise pipeline behind one MCP tool. Keep four jobs in four files:

- `src/agents/drafter.py` — `Draft` (ticket, reply), `run_drafter(model, ticket, feedback=None)`. With feedback, fix ONLY the named issues.
- `src/agents/compliance_checker.py` — `ComplianceVerdict` (`approved`, `issues`), `run_compliance(model, draft)` = `prompt | model.with_structured_output(...)`, with real policy criteria.
- `src/pipeline.py` — `run_pipeline(model, ticket, max_rounds, progress=None)`, a `PipelineResult`, and a `Progress` holder.
- `src/formatting.py` — `format_result` (trace) + `format_partial`.
- `mcp_server.py` — one `draft_safe_reply` tool; the timeout lives **here**, around the whole pipeline call, not inside the Drafter.

Build order: pipeline standalone → wrap as one tool → add the trace → add the timeout + partial.

Hint 2 — the loop, and what makes it real

```
run_pipeline(model, ticket, max_rounds, progress):
    draft = run_drafter(model, ticket)
    for r in 1..max_rounds:
        progress.latest_draft = draft.reply
        v = run_compliance(model, draft)
        progress.rounds.append({round:r, approved:v.approved, issues:v.issues})
        if v.approved: return PipelineResult(..., True, rounds)
        draft = run_drafter(model, ticket, feedback="\n".join(v.issues))
    return PipelineResult(..., False, rounds)    # the CAP
```

Two things make the loop real: the Compliance-Checker's prompt has **real policy criteria** (full refunds need approval; no "I guarantee"; no info leaks — a vague checker approves round 1 and the loop never runs), and the Drafter **fixes only the named issues** (so round 2 differs and it converges).

The `Progress` holder is seeded now for the partial fallback (Step/Hint 4). Test the loop with a fake model (no key): a full-refund promise is rejected then fixed; a never-approved ticket stops at the cap.

Hint 3 — one MCP tool, with a trace

```
mcp = MCPServer("complydesk-service")
@mcp.tool()
async def draft_safe_reply(ticket: str) -> str:
    progress = Progress()
    ... run the pipeline (see Hint 4) ...
    return format_result(result)
```

Keep the tool thin — its job is format/timeout, not policy. `format_result` is a pure function: "Reply for ticket / Compliance trace (per round) / Status / Reply". Log to **stderr** only (stdout is the MCP protocol channel). A client calls one function and gets a policy-safe reply plus a trace of what the checker caught.

Hint 4 — the timeout, and the to_thread trap

The pipeline is **synchronous**; the tool is `async`. Run the pipeline on a worker thread so the timeout can fire:

```
try:
    result = await asyncio.wait_for(
        asyncio.to_thread(run_pipeline, MODEL, ticket, cap, progress),
        timeout=CONFIG.timeout_seconds)
    return format_result(result)
except asyncio.TimeoutError:
    return format_partial(ticket, progress)   # best draft so far
```

The trap: `await asyncio.wait_for(run_pipeline(...))` on a sync function runs it to completion before `wait_for` gets control — blocking the loop, never timing out. `asyncio.to_thread` fixes it. **Known limitation:** `to_thread` can't cancel the worker thread, so the pipeline keeps running after the timeout — note it; a real deployment adds a cancel flag. Prove it with a tiny `REPLY_TIMEOUT_SECONDS` passed via `StdioServerParameters(env=...)` so even a fast run hits the PARTIAL path.

Still stuck? The whole project, every file, is in the complete solution.